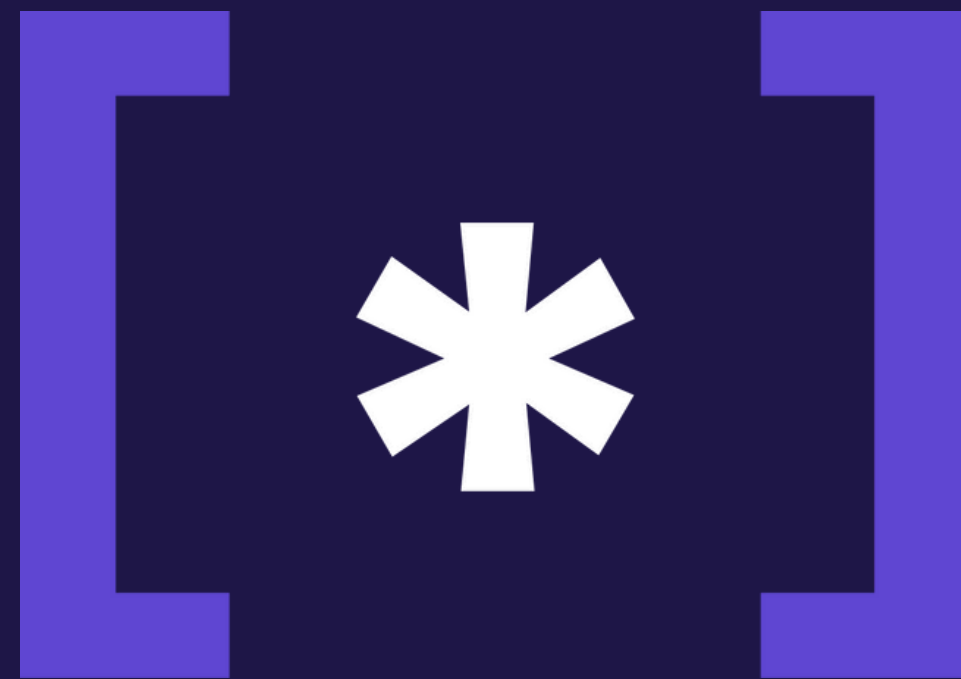
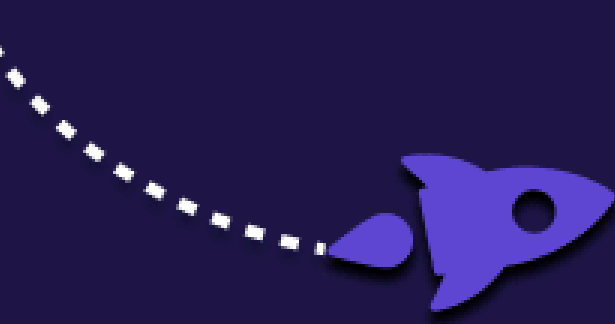




Começaremos em instantes.

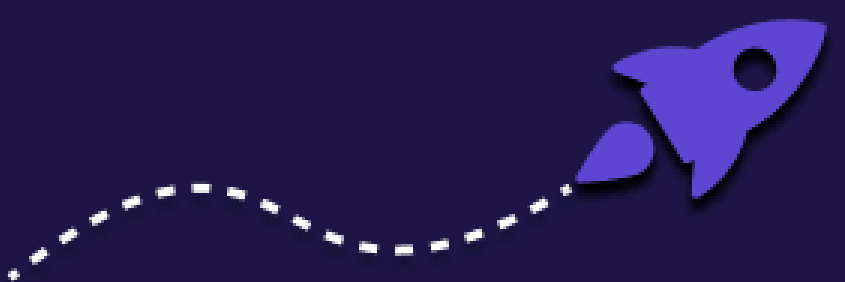
`for_code[ ]`



# MySQL

Banco de Dados Relacionais

Ministrantes: Ana Clara Ruiz e Gabriel Behael



# Apresentação



Gabriel Behael

Pesquisador [IC] na área de nanopartículas  
Diretor de Marketing da for\_code  
Graduando em Engenharia Química

[behaelamorelli@eq.ufrj.br](mailto:behaelamorelli@eq.ufrj.br)

# Apresentação



Ana Clara Ruiz

Estagiária na área de Análise de Dados  
Consultora de projetos na Fluxo  
Ex Diretora de Pessoas na for\_code

[anaruiz@eq.ufrj.br](mailto:anaruiz@eq.ufrj.br)

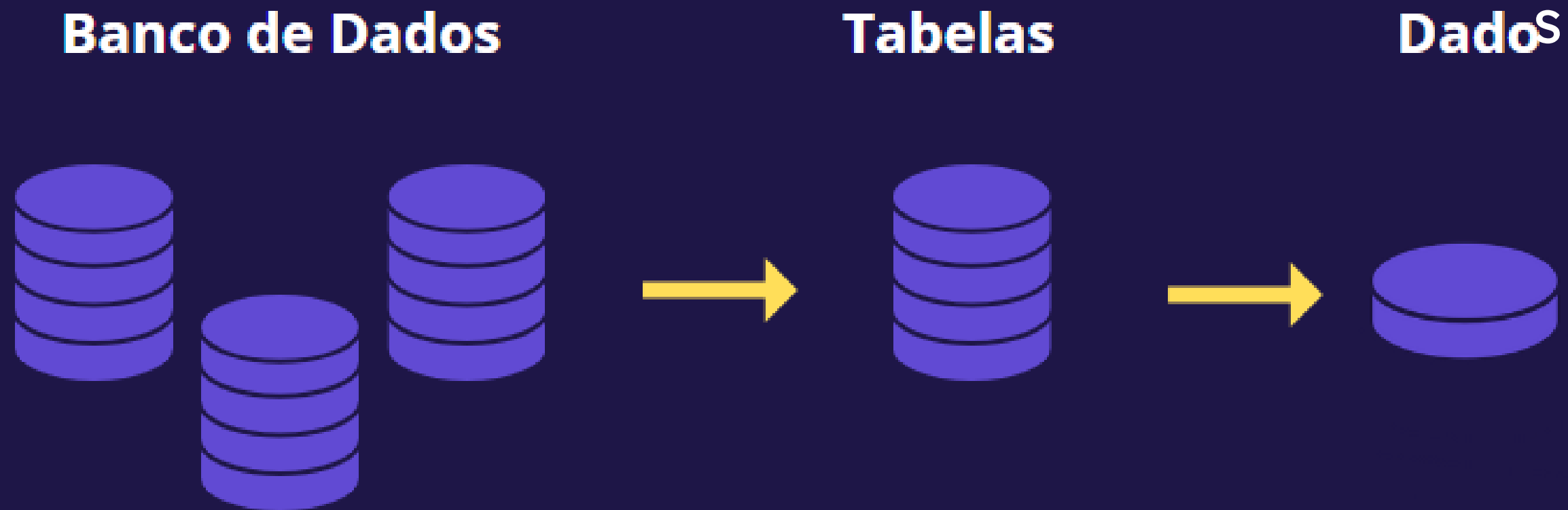
# [ CONTEÚDO ]

- O que é SQL?
- O que é MySQL?
- MySQL X PostgreSQL
- Como instalar o MySQL?
- Criação e Importação de Tabelas
- Criação de filtros
- Funções de Agregação
- Agrupamento
- Relacionamento de tabelas
- Exercícios
- Database, como achar?
- + locais de exercícios



# [ O que são Bancos de Dados? ]

- Bancos de dados são **conjuntos** de tabelas inter-relacionadas que armazenam dados.



# [🚀] O que é SQL?

# O SQL é a linguagem padrão para trabalhar com **bancos de dados relacionais**.

Permite consultar, adicionar e excluir informações em um banco.



# [ SGBDs ]

- Sistemas de Gerenciamento de Banco de Dados e são neles que estarão armazenados esses dados





# [ Query ]

- Formas de **consultar dados** em um banco de dados para manipulação, inserção, atualização, exclusão e outras ações.
- Pedido feito ao banco, com a linguagem devida, para que os dados corretos sejam retornados.

# [ Grupo de comandos ]

## Subconjuntos SQL

DQL

DML

DDL

DCL

DTL

## Comandos SQL

SELECT

INSERT  
UPDATE  
DELETE

CREATE  
ALTER  
DROP

GRANT  
REVOKE

BEGIN  
COMMIT  
ROLLBACK

# [ CRUD ]

SQL

C

R

U

D

CREATE /  
INSERT

READ(SELECT)

UPDATE

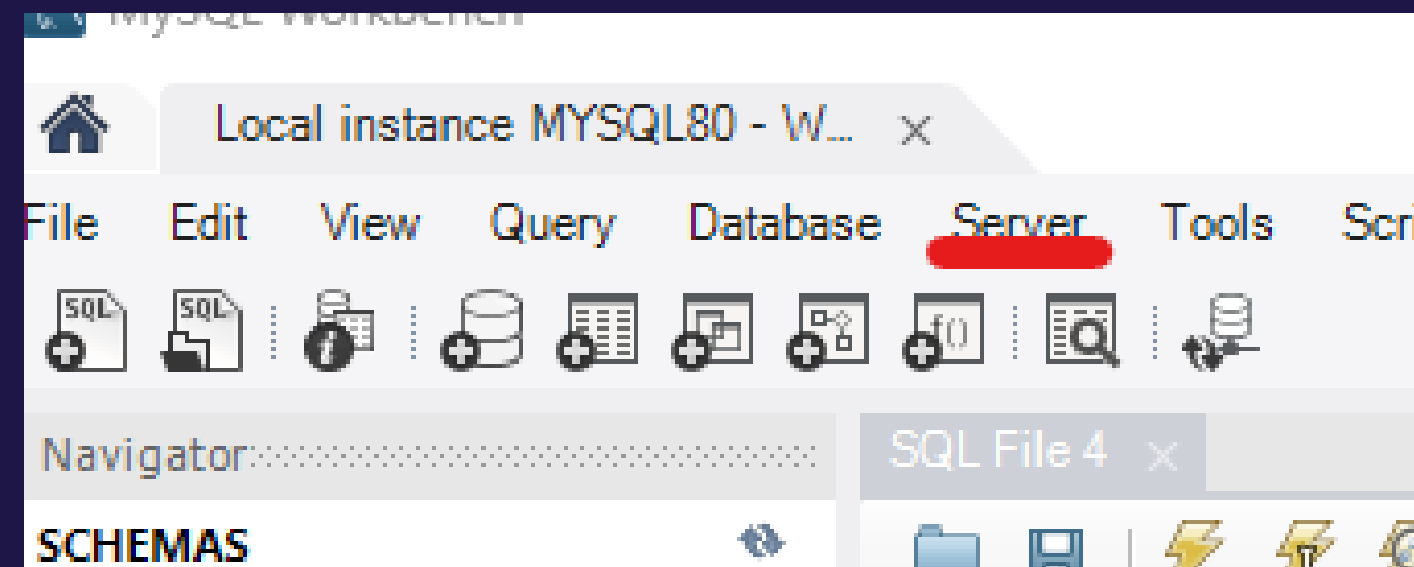
DELETE

for\_code[ ]

[ <#> ]

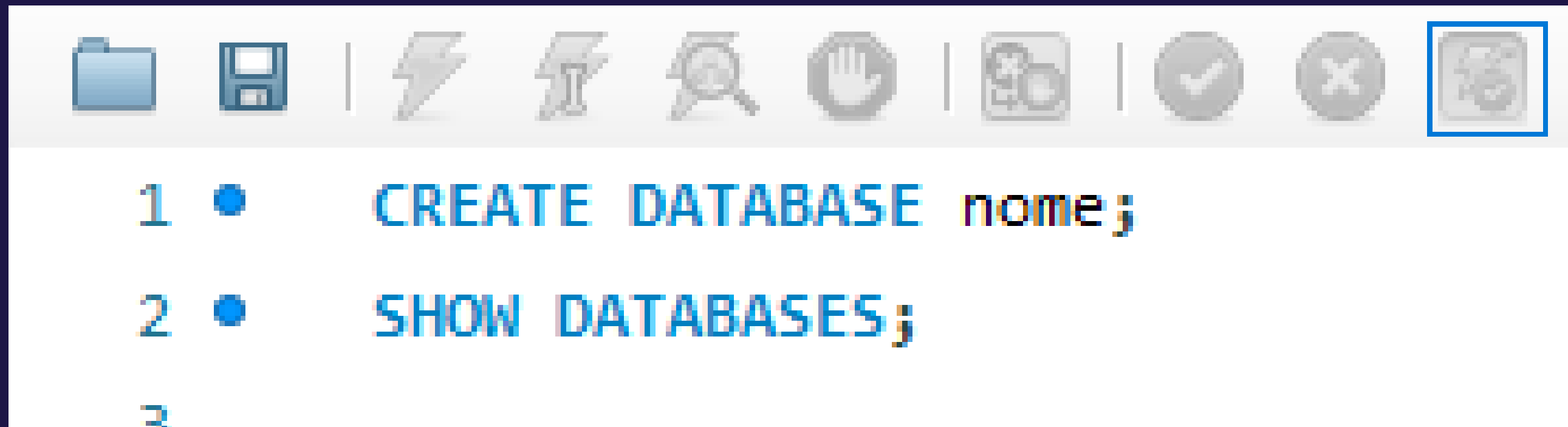
# [ Importação de banco de dados ]

- Feita diretamente pelo MySQL ou pelo terminal.



# [ Criando um banco de dados ]

- A criação de um banco é feita utilizando o seguinte comando:



The image shows a screenshot of a MySQL command-line interface. At the top, there is a toolbar with various icons. The icon for running a query (a green play button) is highlighted with a blue square. Below the toolbar, the command prompt shows two lines of SQL code:

```
1 • CREATE DATABASE nome;  
2 • SHOW DATABASES;  
3
```



# [ Criação de tabelas ]

- Todo banco de dados é formado por tabelas que, conseqüentemente, são formadas por dados.

**CREATE TABLE** <nome> (<coluna><tipo de dados>);

- Nossas tabelas podem conter **diferentes tipos de dados**

# [ Tipos de dados ]

- **CHAR(x)**: aceita textos até 255 caracteres.
- **VARCHAR(x)**: textos até 65536 caracteres.
- **INT(x)**: valores entre -2147483648 a 2147483647;
- **BOOL**: 0 é falso e outros valores verdadeiros.
- **DATE**: aceita uma data no formato YYYY-MM-DD.
- **DATETIME**: aceita uma data com horário no formato YYYY-MM-DD hh:mm:ss.

# [ INSERT ]

- Para inserir dados em uma tabela precisamos inserir o **nome das colunas** e também **os valores** para cada uma ou inserir os dados respectivamente.

```
INSERT INTO <tabela> VALUES (dado1, dado2...);
```



## [ SELECT ]

- Utilizamos este comando para retornar determinados dados da tabela.  
exemplo:

SELECT \* FROM <nome da tabela>

- Cláusula **FROM** especifica de onde os dados virão.

## [ SELECT ]

- **SELECT AS:** permite renomearmos colunas

```
SELECT <coluna> AS <nova_coluna> FROM  
<tabela>;
```

- **SELECT LIMIT:** limita quantas linhas o programa retorna

```
SELECT <dado> FROM <tabela> LIMIT (X);
```

# [ Funções de agregação ]

- Uma função de agregação processa um **conjunto de valores** contidos em uma única coluna de uma tabela e **retorna um único valor** como resultado.

**SELECT** [função(ões) de agregação/coluna(s)] **FROM**  
[tabela(s)]

- MAX, MIN, SUM, COUNT e DISTINCT

# [ WHERE ]

- Com ele, podemos **definir uma condição** e só serão exibidas as informações que corresponderem a ela.

**SELECT** <dado> **FROM** <tabela> **WHERE** <condição>;

- Não restrito ao SELECT

# [ Operadores ]

|    |                |
|----|----------------|
| =  | Igual          |
| <  | Menor          |
| <= | Menor ou igual |
| >  | Maior          |
| >= | Maior ou igual |
| <> | Diferente      |

|         |                                    |
|---------|------------------------------------|
| AND     | Todas condições verdadeiras        |
| OR      | Pelo menos uma condição verdadeira |
| NOT     | Não satisfaz a condição            |
| BETWEEN | Dentro de um intervalo             |
| LIKE    | Pesquisar por padrão               |
| IN      | Intervalo de valores               |



# [ Alteração de tabelas ]

- Adição de coluna:

ALTER TABLE <tabela> ADD COLUMN <nome><tipo>

- Remoção de coluna:

ALTER TABLE <tabela> DROP COLUMN <nome>

- Modificar tipo de coluna:

ALTER TABLE <tabela> MODIFY COLUMN <coluna>  
<tipo>

## [ UPDATE ]

- Realiza **alterações nos dados** armazenados em uma tabela do banco de dados.

UPDATE nome\_da\_tabela SET coluna = valor  
WHERE condição;

## [ DELETE ]

- Realiza a **exclusão de dados** de uma ou mais tabelas de um banco de dados.

**DELETE FROM** nome\_da\_tabela **WHERE**  
condição;



# [ CHAVES ]



- Primária, Alternativa e Estrangeira
- São colunas
- Importante para o conceito de Tabela **Relacional**
- Pode haver regra de chave primária composta e simples

# [ CHAVES: primary key ]

## Primária

Identificador único de linhas

Apenas uma por tabela

Serve como índice interno

Garantia de que o dado existe

Valores ÚNICOS NÃO NULOS

Simples: Um único campo ou coluna



| Chave | CPF    | Aluno  |
|-------|--------|--------|
| 1     | 5555-Y | Pedro  |
| 2     | 2222-N | Ana    |
| 3     | 6666-I | João   |
| 4     | 9999-O | Felipe |
| 5     | 8888-X | Maria  |
| 6     | 9999-Y | Teresa |
| 7     | 6666-X | Carlos |

Composto: dois ou mais campos ou colunas



| Chave | CPF    | Aluno  |
|-------|--------|--------|
| 1     | 5555-Y | Pedro  |
| 2     | 2222-N | Ana    |
| 3     | 6666-I | João   |
| 4     | 9999-O | Felipe |
| 5     | 8888-X | Maria  |
| 6     | 9999-Y | Teresa |
| 7     | 6666-X | Carlos |

primary key (cpf);

ou

nome\_coluna formato AUTO\_INCREMENT PRIMARY KEY;

# [ CHAVES: foreign key ]

## Estrangeira

Relaciona as linhas da tabela à outra tabela

É armazenado o valor da chave primária desejável de ligação

Garante integridade referencial de dados

Nome pode diferir da primária, mas deve ter mesmo tipo de dados

Pode ter repetição

| Id_pai | CPF    | Aluno  |
|--------|--------|--------|
| 1      | 5555-Y | Pedro  |
| 2      | 2222-N | Ana    |
| 3      | 6666-I | João   |
| 4      | 9999-O | Felipe |
| 5      | 8888-X | Maria  |
| 6      | 9999-Y | Teresa |
| 7      | 6666-X | Carlos |

Tabela Pai

Chave primária

| Chave | Id_filho | Aluno  |
|-------|----------|--------|
| 1     | 1        | Carlos |
| 2     | 1        | Maria  |
| 3     | 1        | Ana    |
| 4     | 2        | Joana  |
| 5     | 2        | Felipe |
| 6     | 3        | Teresa |
| 7     | 4        | Carlos |

Tabela Filho

Chave estrangeira

**FOREIGN KEY (id-filho) REFERENCES TAB\_PAIS (id\_pai);**

| Produtos                                                                                     |                                                                                             |
|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
|  id_produto | INTEGER                                                                                     |
| nome                                                                                         | TEXT                                                                                        |
| fornecedor                                                                                   | TEXT                                                                                        |
| id_membro_contatante                                                                         | INTEGER  |
| quantidade                                                                                   | INTEGER                                                                                     |
| preço                                                                                        | INTEGER                                                                                     |

| Membros                                                                                        |                                                                                             |
|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
|  id_membros | INTEGER  |
| membro                                                                                         | TEXT                                                                                        |
| idade                                                                                          | INTEGER                                                                                     |
| gênero                                                                                         | TEXT                                                                                        |
| diretoria                                                                                      | TEXT                                                                                        |
| id_projeto                                                                                     | INTEGER  |

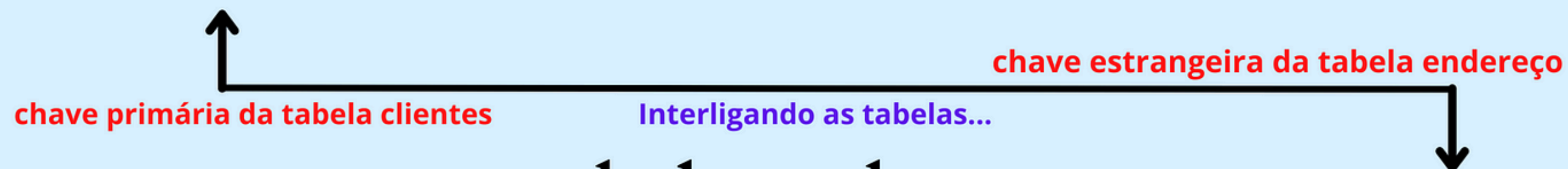
| Pesquisa_Satisfação |                                                                                               |
|---------------------|-----------------------------------------------------------------------------------------------|
| ordem_preenchimento | INTEGER  |
| data                | DATE                                                                                          |
| diretoria           | TEXT                                                                                          |
| descrição           | TEXT                                                                                          |

| Apadrinhamento                                                                               |      |
|----------------------------------------------------------------------------------------------|------|
|  Membro | TEXT |
| Diretoria_membro                                                                             | TEXT |
| Padrinho                                                                                     | TEXT |
| Diretoria_padrinho                                                                           | TEXT |

| Projetos                                                                                         |         |
|--------------------------------------------------------------------------------------------------|---------|
|  id_projeto | INTEGER |
| nome_projeto                                                                                     | TEXT    |
| quantidade_membros                                                                               | INTEGER |
| project_owner                                                                                    | TEXT    |
| tempo_estimado_mes                                                                               | INTEGER |

## tabela clientes

| ID<br>CLIENTES | NOME    | CPF                | EMAIL          |
|----------------|---------|--------------------|----------------|
| 01             | Miranha | 111.222.111.666-00 | miranha@spider |



## tabela endereco

| ID<br>endereço | RUA             | NUMERO | BAIRRO         | ID<br>CLIENTES |
|----------------|-----------------|--------|----------------|----------------|
| 10             | rua das paredes | 07     | ratao@rato.com | 01             |



# [ TABELAS ]

- 2 tipos: Fato e Dimensão
- Durante a análise dos dados, é pertinente trazer informações de uma tabela para outra

## Dimensão

Onde são encontradas as chaves primárias

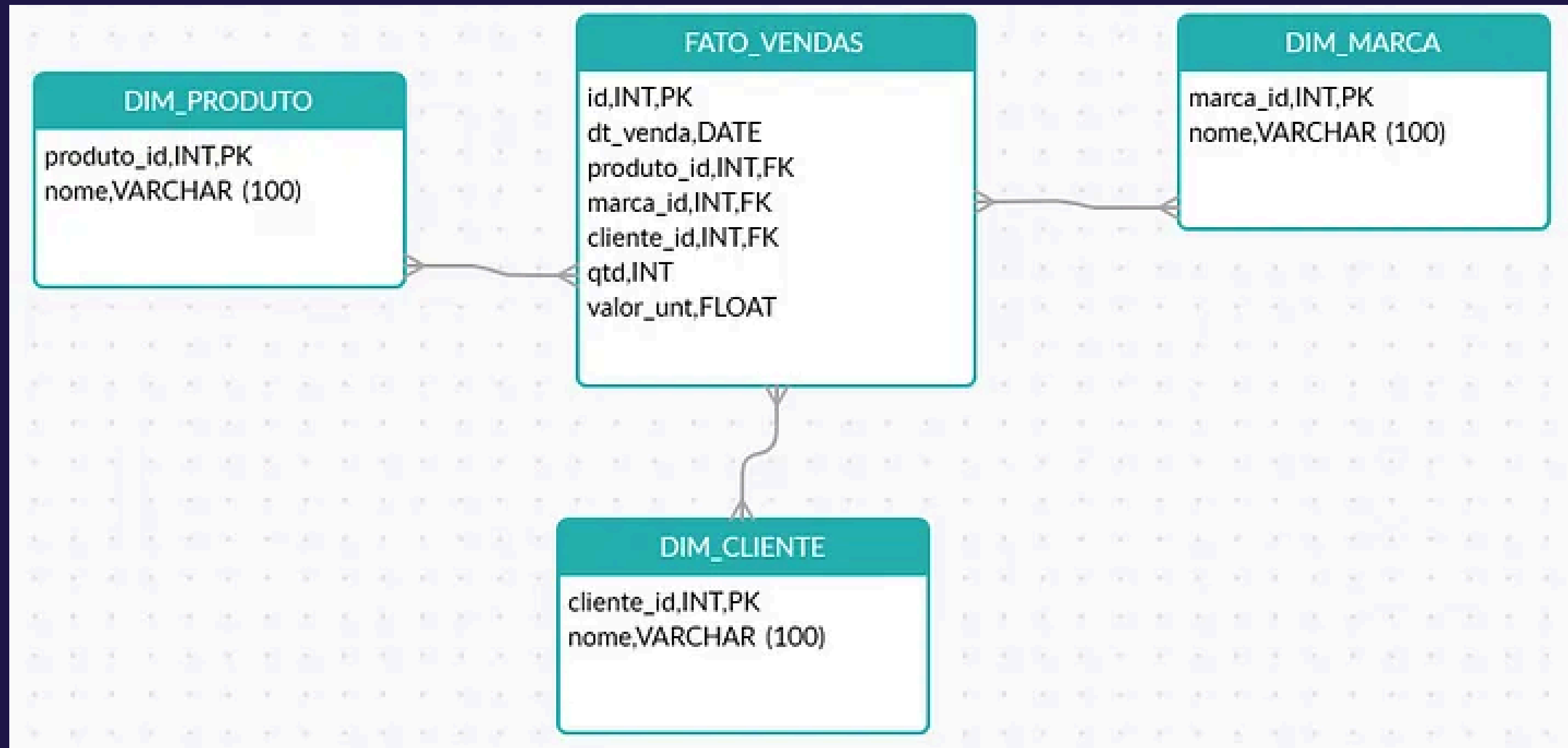
Características sobre alguma informação

## Fato

Muitas informações que podem se repetir

Onde são encontradas as chaves estrangeiras

# [ TABELAS ]



# [ GROUP BY ]

- Agrupamento de linhas
- Baseia-se em semelhanças
- Permite a criação de tabelas-resumo
- Exemplos:
  - Perfil de clientes;
  - Perfil de produtos.

| id | fruit  |
|----|--------|
| 1  | Apple  |
| 2  | Orange |
| 3  | Apple  |
| 4  | Banana |
| 5  | Orange |



| fruit  |
|--------|
| Apple  |
| Banana |
| Orange |

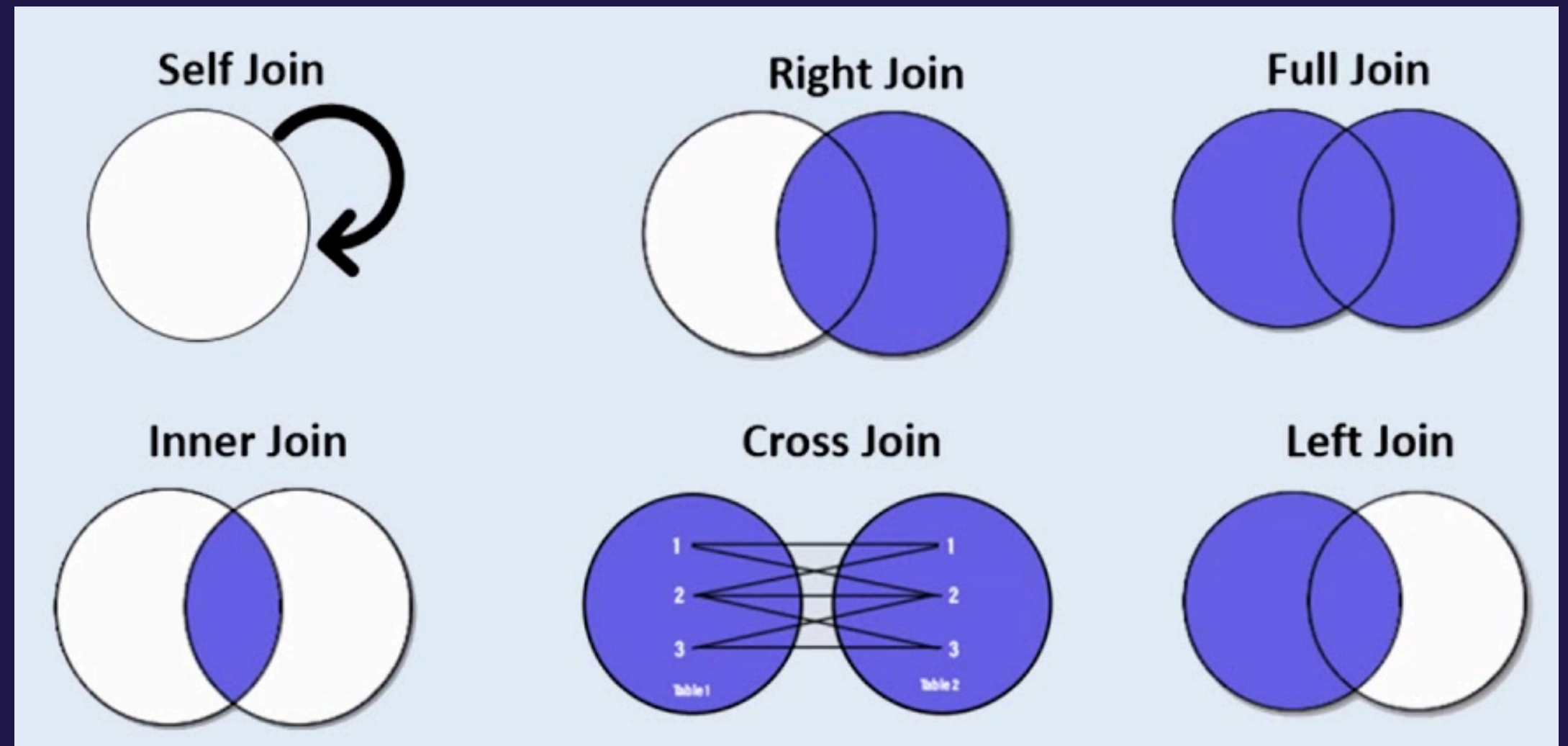
## SINTAXE

```
SELECT <colunaA> FROM  
<tabelas>  
WHERE <condição>  
GROUP BY <colunaA>  
HAVING <coluna>;
```

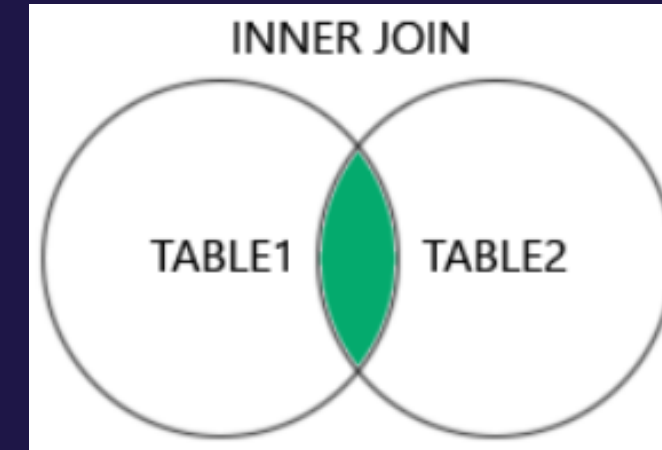


# [ JOIN ]

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- CROSS JOIN
- FULL JOIN



# [ INNER JOIN ]



- O que faz?
  - Relaciona 2 tabelas (A e B) criando uma nova (C)
  - Retorna um match das 2 tabelas

- Sintaxe

```
SELECT <tabelaA.coluna1>, <tabelaB.coluna2>  
FROM <tabelaA  
INNER JOIN <tabelaB>  
ON <tabelaA.nome_coluna> = <tabelaB.nome_coluna>;
```

## INNER JOIN

**Customers**

| CustomerID | Name | CountryID |
|------------|------|-----------|
| 1          | Leo  | 2         |
| 2          | Zion | 4         |
| 3          | Ivy  | 1         |
| ...        | ...  | ...       |

**Orders**

| OrderID | CustomerID | OrderDate  |
|---------|------------|------------|
| 1       | 11         | 2018-03-06 |
| 2       | 11         | 2018-04-11 |
| 3       | 2          | 2019-05-17 |
| ...     | ...        | ...        |

**Countries**

| CountryID | CountryName |
|-----------|-------------|
| 2         | Canada      |
| 3         | Egypt       |
| 4         | Brazil      |
| ...       | ...         |

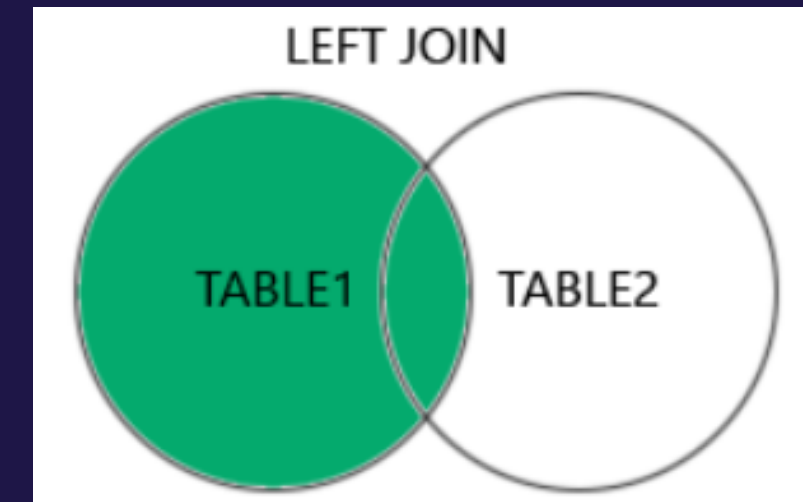
INNER JOIN on CustomerID column

RESULT

| CustomerID | Name | OrderID | OrderDate  |
|------------|------|---------|------------|
| 2          | Zion | 3       | 2019-05-17 |
| 5          | Luca | 4       | 2018-12-06 |
| 1          | Leo  | 5       | 2019-02-27 |
| 2          | Zion | 6       | 2020-01-29 |
| 2          | Zion | 7       | 2018-08-16 |

\*customerdemo database

# [ LEFT JOIN ]



- O que faz?
  - Relaciona 2 tabelas (A e B) criando uma nova (C)
  - Retorna a tabela A + resultados A=B

- Sintaxe

```
SELECT <nome_coluna(s)>  
FROM <tabelaA>  
LEFT JOIN <tabelaB>  
ON <tabelaA.nome_coluna> = <tabelaB.nome_coluna>;
```



| Employees  |              |          |
|------------|--------------|----------|
| EmployeeID | EmployeeName | GenderID |
| 1          | Mark         | 1        |
| 2          | Sara         | 2        |
| 3          | Tom          | NULL     |

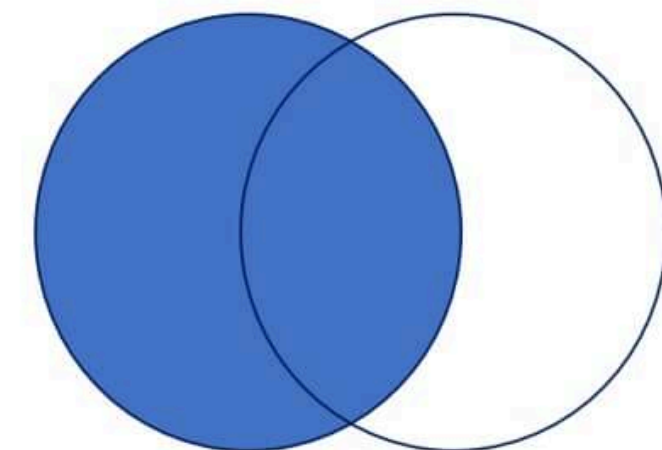
| Genders  |               |
|----------|---------------|
| GenderID | Gender        |
| 1        | Male          |
| 2        | Female        |
| 3        | Not Specified |

```
SELECT EmployeeID, EmployeeName, Gender
FROM Employees LEFT JOIN Genders
ON Employees.GenderID = Genders.GenderID
```

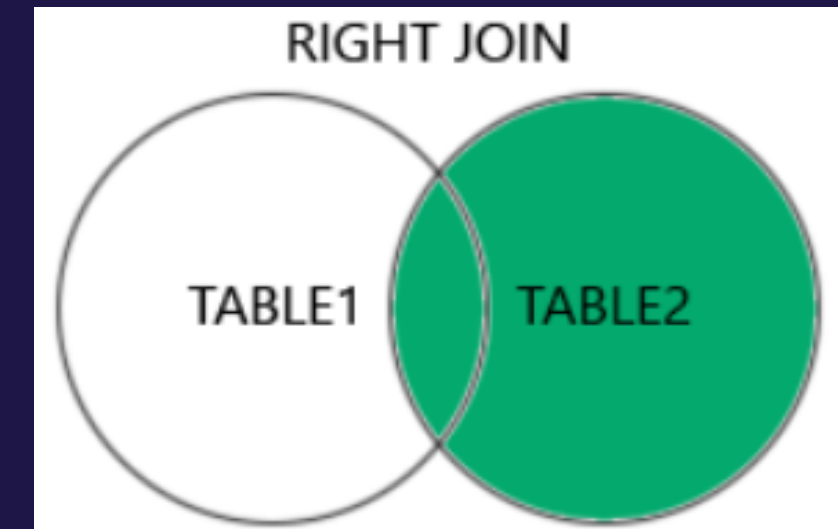
**LEFT JOIN  
or  
LEFT OUTER JOIN**

Query Result

| EmployeeID | EmployeeName | Gender |
|------------|--------------|--------|
| 1          | Mark         | Male   |
| 2          | Sara         | Female |
| 3          | Tom          | NULL   |



# [ RIGHT JOIN ]



- O que faz?
  - Relaciona 2 tabelas (A e B) criando uma nova (C)
  - Retorna a tabela B + resultados A=B
- Sintaxe

```
SELECT <nome_coluna(s)>  
FROM <tabelaA>  
RIGHT JOIN <tabelaB>  
ON <tabelaA.nome_coluna = <tabelaB.nome_coluna>;
```

| Employees  |              |          | Genders  |               |
|------------|--------------|----------|----------|---------------|
| EmployeeID | EmployeeName | GenderID | GenderID | Gender        |
| 1          | Mark         | 1        | 1        | Male          |
| 2          | Sara         | 2        | 2        | Female        |
| 3          | Tom          | NULL     | 3        | Not Specified |

SELECT EmployeeID, EmployeeName, Gender  
FROM Employees RIGHT JOIN Genders  
ON Employees.GenderID = Genders.GenderID

RIGHT JOIN  
or  
RIGHT OUTER JOIN

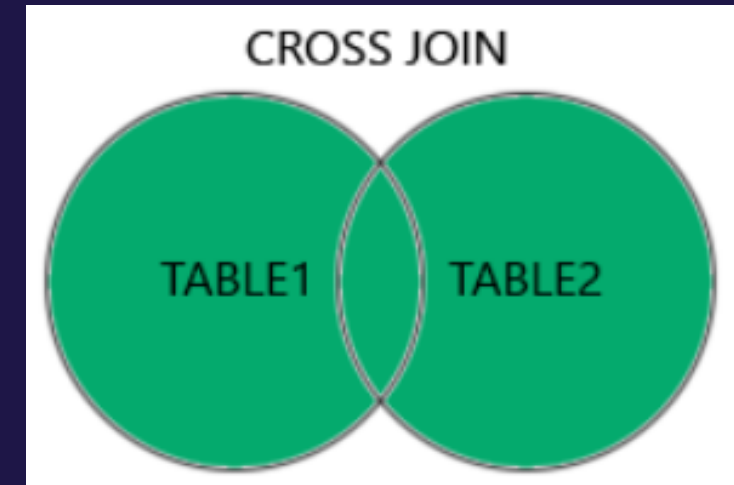
  

Query Result

| EmployeeID | EmployeeName | Gender        |
|------------|--------------|---------------|
| 1          | Mark         | Male          |
| 2          | Sara         | Female        |
| NULL       | NULL         | Not Specified |



# [ CROSS JOIN ]

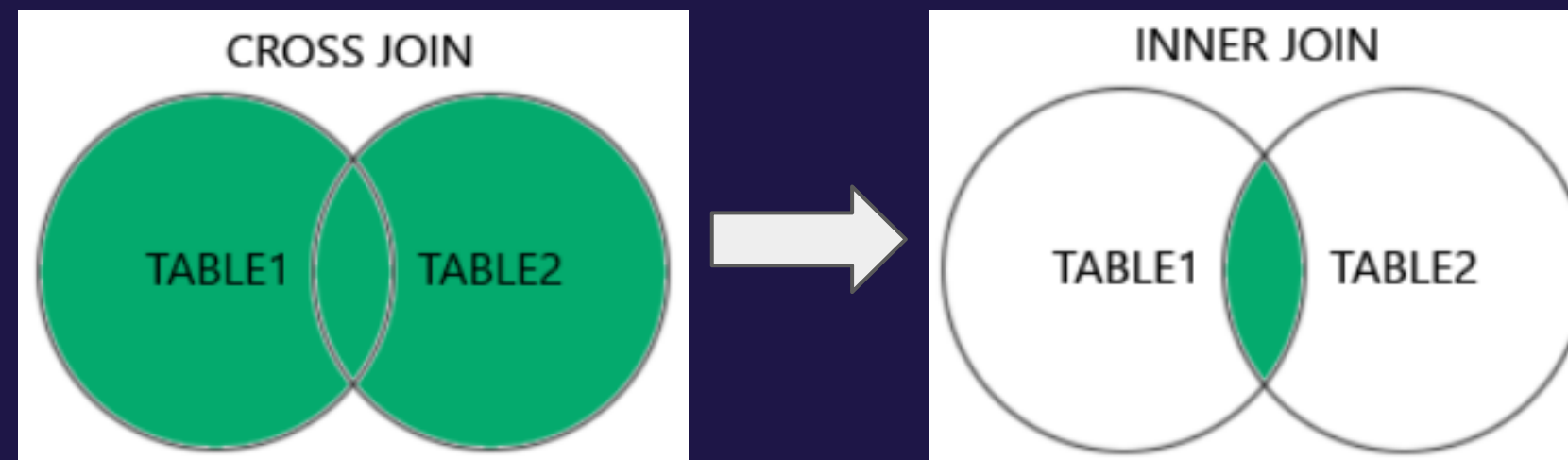


- O que faz?
  - Retorna TODOS os dados de ambas tabelas
- Sintaxe

```
SELECT <nome_coluna(s)>  
FROM <tabelaA>  
CROSS JOIN <tabelaB>;
```



# [ CROSS JOIN ]



É possível adicionar uma cláusula WHERE que produz o mesmo resultado do INNER JOIN:

**WHERE** <tabelaA.id\_chave\_estrangeira> = <tabelaB\_id\_chave\_primaria>

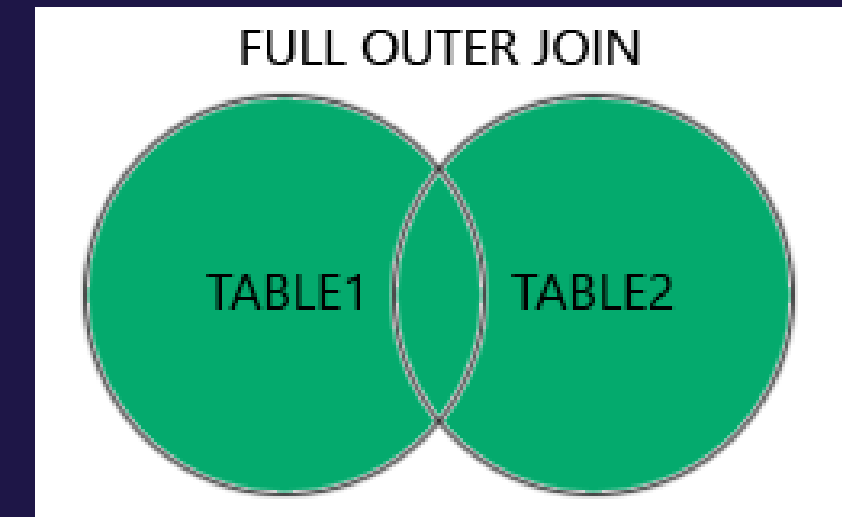
| Meals     |
|-----------|
| Omlet     |
| Fried Egg |
| Sausage   |

| Drinks       |
|--------------|
| Orange Juice |
| Tea          |
| Coffee       |

**CROSS JOIN**

| MealName  | DrinkName    |
|-----------|--------------|
| Omlet     | Orange Juice |
| Fried Egg | Orange Juice |
| Sausage   | Orange Juice |
| Omlet     | Tea          |
| Fried Egg | Tea          |
| Sausage   | Tea          |
| Omlet     | Coffee       |
| Fried Egg | Coffee       |
| Sausage   | Coffee       |

# [ FULL JOIN ]

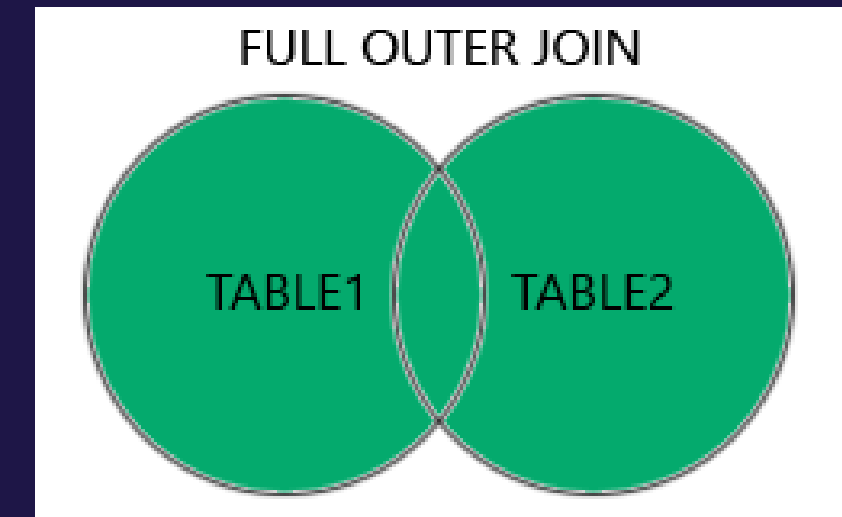


- O que faz?
  - União de LEFT e RIGHT JOIN
  - Mostra as linhas correspondentes e não correspondentes das 2 tabelas
  - Nem todas as versões do SQL suportam o FULL JOIN
- Sintaxe

```
SELECT <nome_coluna(s)>  
FROM <tabelaA>  
FULL JOIN <tabelaB> (ou FULL OUTER JOIN)  
ON <tabelaA.nome_coluna = <tabelaB.nome_coluna>  
WHERE <condição>;
```

[https://www.w3schools.com/sql/sql\\_join.asp](https://www.w3schools.com/sql/sql_join.asp)

# [ UNION ]



- Sintaxe

```
SELECT <nome_coluna(s)>
FROM <tabelaA>
RIGHT JOIN <tabelaB>
ON <tabelaA.nome_coluna = <tabelaB.nome_coluna>
UNION
SELECT <nome_coluna(s)>
FROM <tabelaA>
LEFT JOIN <tabelaB>
ON <tabelaA.nome_coluna = <tabelaB.nome_coluna>
```

[https://www.w3schools.com/sql/sql\\_join.asp](https://www.w3schools.com/sql/sql_join.asp)

| table_1 |
|---------|
| 1       |
| 2       |
| 3       |
| 4       |

**FULL JOIN**

| table_2 |
|---------|
| A       |
| B       |
| C       |
| D       |

=

|      |      |
|------|------|
| 1    | A    |
| 2    | C    |
| 3    | D    |
| 4    | NULL |
| NULL | B    |



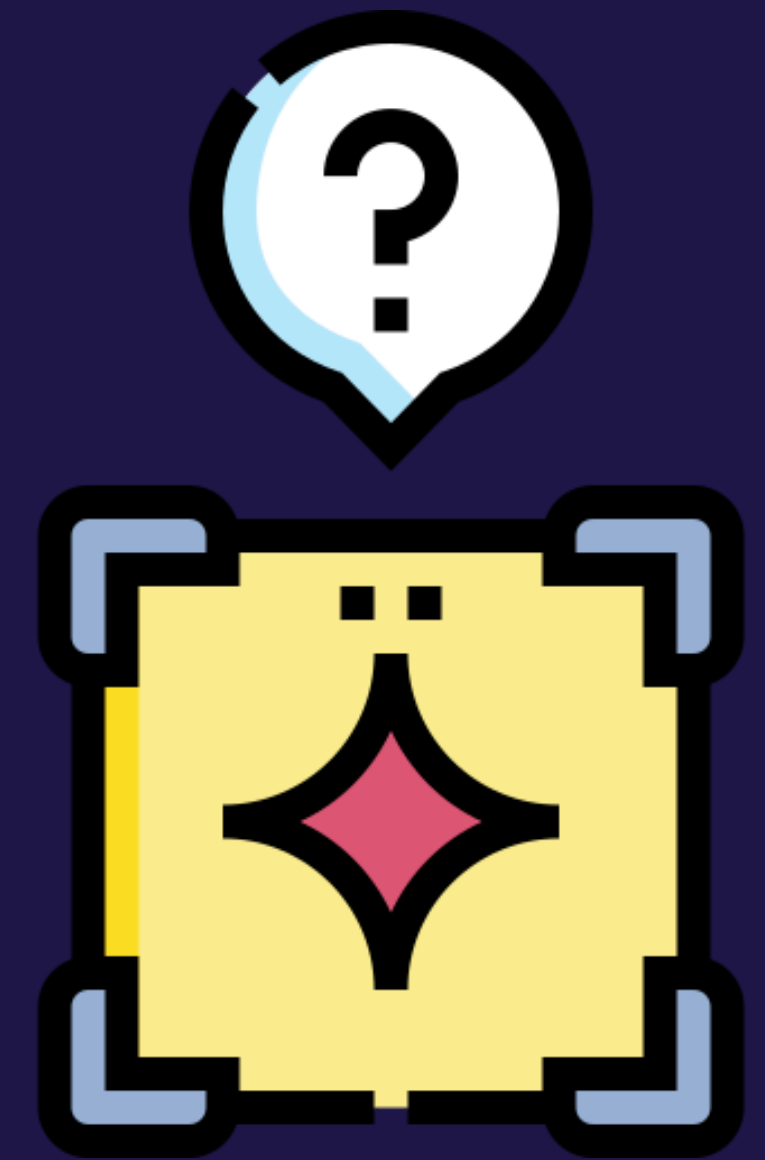
# [ Encontrando Databases ]



- <https://www.kaggle.com/>
- <https://dados.gov.br/dataset>
- <https://data.europa.eu/euodp/en/home>
- <https://www.who.int/data/gho>
- <https://www.semanticscholar.org/cord19>
- <https://www.imf.org/en/Data>
- <http://ai.stanford.edu/~amaas/data/sentiment/>

# [ JOGOS ]

- Mistérios:
  - <https://mystery.knightlab.com/>
- Treino:
  - <https://web.sqlplay.net/>
- Vendo como estruturar:
  - <https://www.w3schools.com/sql/default.asp>



# [ APLICAÇÕES ]

- Conhecimento valioso para BACK-END
  - MER
  - [O que é MER? \(lucidchart\)](#)
  - Aplicativos
  - Sites
- Sistemas Corporativos
- ....

